Hertie School

Natural Language Processing

Session 2: String Processing,
Regular Expressions & Preprocessing

GRAD-E1282 · Fall Semester 2026
Luis Fernando Ramírez Ruiz ·
l.ruiz@hertie-school.org

Where we left off — and where we start today

Session 1 ended with a chain of choices. Session 2 opens the first link in it.

1 · SESSION 1 STOPPED HERE

`raw text`

not yet computable

2 · WHAT TODAY ADDS

`strings`
`regex`
`tokens`
`normalization`

the first layer of the pipeline

3 · WHERE IT LEADS

`LANGUAGE → REPRESENTATION → MODEL → EVALUATION`

Session 3 picks up at representation

Session 1 asked what NLP is. Session 2 asks what a unit of text is.

strings · regex · tokenization · normalization

Today: from raw text to usable units

Hertie School

RAW TEXT → **STRING OPS** → **REGEX** → **TOKENS** → **NORMALIZE**

characters → inspect / clean → describe patterns → choose units → standardize

BY THE END OF TODAY

1. Read and construct useful regular expressions for common text patterns.
2. Explain why tokenization and normalization are modeling choices, not neutral cleanup.
3. Recognize where preprocessing can create false positives, false negatives, or information loss.

Strings are sequences, not meaning

A Python string is an ordered sequence of characters.

```
"Room 42, Deck 7"
```

R	o	o	m		4	2	,		D	e	c	k		7
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Each character has a position; slicing selects positions, not “meaning”.

SAME IDEA AT WORD LEVEL

"policy" ——— p → o → l → i → c → y

order matters: p-o-l-i-c-y ≠ y-c-i-l-o-p

USEFUL OPERATIONS

slice	`text[0:4]  →  "Room"`
split	`text.split(", ")`
case	`text.lower()`
replace	`text.replace("Deck", "D")`

String operations are exact. They work well when we know the exact characters we want to inspect or change.

From string operations to regular expressions

Hertie School

The problem is variation: natural text rarely arrives in one exact form.

1 · STRING OPERATIONS

```
text == "Hi"
```

asks about one literal string

2 · TEXT VARIES

"Hi"
"Hii"
"Hiii!"
"Hello,"
"Hey"

same intent, different characters

3 · REGEX PATTERN

```
r"(hi+|hello|hey)[ ,:.!]*"
```

describes a family of acceptable strings

Regex generalizes string operations from exact strings to sets of strings.

Jurafsky: “a regular expression is an algebraic notation for characterizing a set of strings”

search · extract · validate · normalize

Regex basics: what can match, and how many times?

Hertie School

MATCH / POSITION

`abc`	literal sequence
`[abc]`	one character from the set
`[^abc]`	one character not in the set
`.`	any single character
`\d  \w  \s`	digit · word character · whitespace
`^  $  \b`	start · end · word boundary

COMBINE / REPEAT

`|`	or / alternative
`(...)`	group; also captures by default
`?`	0 or 1 occurrence of the char expression
`*`	0 or more occurrences of the char expression
`+`	1 or more occurrences of the char expression
`{n,m}`	between n and m repetitions

PYTHON re: MAIN COMMANDS

`re.search()`	`re.findall()`	`re.sub()`	`re.split()`	`re.match()`
first match anywhere	all non-overlapping matches	replace matches	split on a pattern	match only at the start

Reading a regex from left to right

Hertie School

ONE COMPLETE EXAMPLE

```
^(cat|dog)s?$
```

`^`	→	`(cat|dog)`	→	`s?`	→	`$`
START		**CHOICE**		**OPTIONAL**		**END**
start of string		cat OR dog		s appears 0 or 1 time		end of string

Plain English: "The entire string must be cat or dog, optionally followed by s."

MATCHES

✓ `cat` ✓ `cats` ✓ `dog`
✓ `dogs`

DOES NOT MATCH

✗ `a cat` ✗ `hotdog` ✗ `dog park`

`\b ≠ whitespace`

\b means "word boundary". To find cat/cats/dog/dogs inside a longer sentence, use \b(cat|dog)s?\b instead of ^ … $.

Three Patterns as Examples

Hertie School

For each task: write a regex, then invent one false positive and one false negative.

1 DATE

```
r"\b\d{2}/\d{2}/\d{4}\b"
```

text = "briefing on 17/09/2026"

two digits / two digits / four digits

RESULT: `17/09/2026`

2 EMAIL

```
r"[\w.-]+@[\w.-]+\.\w+"
```

text = "write to press.office@gov.example"

local part + @ + domain + literal dot

RESULT: `press.office@gov.example`

3 QUOTED TEXT

```
r'"[^"]*"'
```

text = 'He said "Engage" and then "Make it so."'

match characters until the next quote

RESULT:
```
"
Engage"
"Make it so."
```

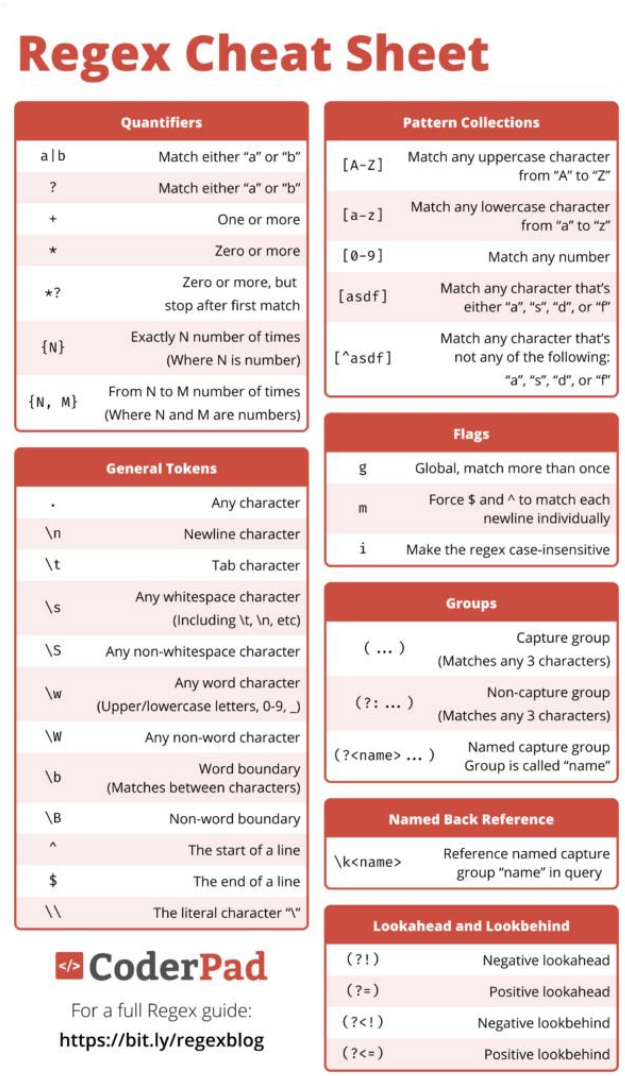
5 min in pairs → 3 min discussion

Exercise: design / find a pattern

For each task: write a regex, then invent one false positive and one false negative.

1. **Find the article “the”** — Match The / the, but not other or theology.
2. **Find prices** — Match $199 and $199.99 without swallowing nearby text.
3. **Find storage mentions** — Match 500 GB, 5.5 GB, and 500 gigabytes.

5 min in pairs → 3 min discussion



Regex is already an evaluation problem

Hertie School

Goal: find the English article “the”.

`/the/`	misses “The”	**false negative**
`/[tT]he/`	also matches “other”	**false positive**
`/\b[tT]he\b/`	better boundary control	**fewer errors**

PRECISION

Of what we matched, how much was actually relevant?

TP / (TP + FP)

RECALL

Of what was relevant, how much did we manage to match?

TP / (TP + FN)

Regex can transform text, not only find it

Hertie School

One pattern language, two jobs: **re.search** finds a span, **re.sub** rewrites it.

1 · STANDARDIZE

Collapse surface variants into one form.

```
pattern   r"colou?r"
replace   "color"
```

BEFORE

```
"the colour of the report"
```

AFTER

```
"the color of the report"
```

One rule replaces a whole family of spellings.

2 · RESTRUCTURE

Capture groups remember what matched.

```
pattern   r"(\d{2})/(\d{2})/(\d{4})"
replace   r"\3-\2-\1"
```

BEFORE

```
"briefing on 17/09/2026"
```

AFTER

```
"briefing on 2026-09-17"
```

\1 \2 \3 refer back to the groups, in any order.

3 · REMOVE

Delete or mask spans you must not keep.

```
pattern   r"[\w.-]+@[\w.-]+\.\w+"
replace   "<EMAIL>"
```

BEFORE

```
"write to press.office@gov.example"
```

AFTER

```
"write to <EMAIL>"
```

The same move strips HTML tags when scraping.

Standardize, restructure, or remove — regex edits the text before any model sees it.

Preprocessing and Text Normalization = deliberate information loss

Hertie School

OPERATION	EXAMPLE	WHAT WE GAIN	WHAT WE MAY LOSE
case folding	Woodchuck → woodchuck	fewer surface variants	US vs us; names / style
stemming	houses → hous	broader matching	over- or under-stemming
lemmatization	is / are → be	shared dictionary form	sense distinctions / extra assumptions
sentence split	text → sentences	usable local units	abbreviation ambiguity: Mr. / Inc.

Ask before normalizing: which distinctions can this task safely forget?

Tokenization is a choice, not a fact

Hertie School

> “The San Francisco-based restaurant,” they said, “doesn’t charge $10.”

San Francisco-based	one token, three words, or something else?
doesn’t	one token, or does + n’t?
$10	one unit, or currency symbol + number?
punctuation	discard it, attach it, or tokenize it separately?

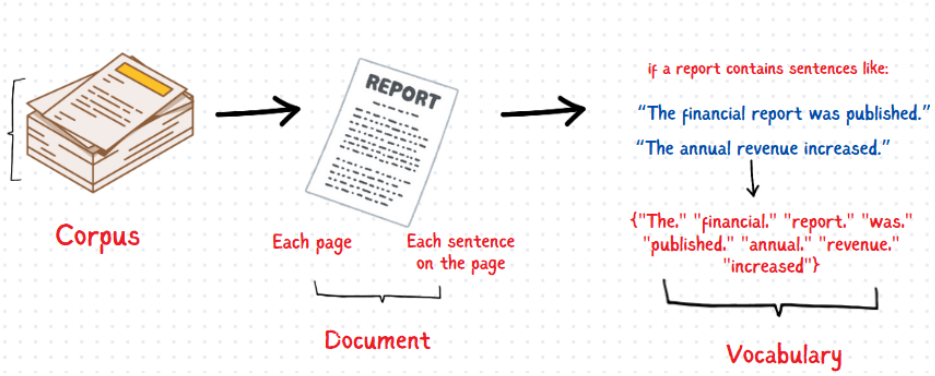
There is no universally “correct” tokenization. The right unit depends on the downstream task.

Session Break

When we return: transformation, tokenization, and preprocessing choices.

Main ideas for the next sessions:

1. **Corpus** — Collection of texts used for our analysis.
2. **Documents** — Self-contained unit of information within the corpus.
3. **Token** — Unit of analysis for our corpus.
3. **Vocabulary** — Set of unique words/tokens in our corpus.



Different tokenizers preserve different information

Hertie School

INPUT	`There's no such thing as survival of the fittest.`	
str.split()	`["There's", "no", …, "fittest."]`	Fast; punctuation stays attached.
regex tokenizer	`["There's", "no", …, "fittest", "."]`	Rules decide what internal punctuation to keep.
spaCy	`["There", "'s", "no", …, "fittest", "."]`	Adds linguistic conventions and token attributes.

Tokenizer = information filter. When a pipeline behaves strangely, inspect the tokens and vocabulary first.

When words are not enough: subword tokenization

Hertie School

Problem Training sees low, new, newer. Test time contains lower.

Start with small units

`l o w e r` —— learn frequent merges —— reuse known pieces —— `low + er`

Why this matters Unknown or rare words can still be represented as familiar pieces instead of collapsing to one OOV token.

Tools you will encounter **SentencePiece · Hugging Face tokenizers**

How BPE learns a vocabulary: merge the frequent pair

Byte-Pair Encoding builds its vocabulary bottom-up: start from characters, then repeatedly merge the most frequent adjacent pair.

TRAINING CORPUS

low_	5
lowest_	2
newer_	6
wider_	3
new_	2

Word counts, each word split into characters. _ marks the end of a word.

STEP 1

Start from characters

l o w e s t _

Initial vocabulary = every character in the corpus. No word is unknown, because any word can be spelled out.

STEP 2

Count adjacent pairs

e + r → 9x

Scan the corpus and count every adjacent symbol pair. Here e r is the most frequent, appearing in newer and wider.

STEP 3

Merge the winner

e r → er

Add er to the vocabulary and rewrite the corpus with it. One merge rule is recorded, in order.

STEP 4

Repeat k times

er _ → er_

k is the only real hyperparameter: it sets the vocabulary size. Later merges build longer pieces such as new, low, est_.

At test time

The learned merges are applied in the order they were learned. An unseen word is segmented into pieces the model already knows.

lower_ → low | er_

never seen in training, still fully representable

BPE never asks what a word means, only which character sequences repeat often enough to earn a vocabulary slot.

BPE, WordPiece, Unigram: three ways to choose pieces

All three learn subwords from a corpus. They differ in the criterion that decides which piece is worth keeping.

BPE

Greedy frequency

```
merge argmax  count(x, y)
```

Criterion

the pair that occurs most often

Direction

bottom-up: grow from characters

Marking

byte-level variants avoid unknown characters

Used by

GPT-2, GPT-4, RoBERTa, Llama

Easy to implement and to explain; frequency alone can favour common but uninformative pieces.

WordPiece

Likelihood gain

```
merge argmax  count(x, y)
       count(x) · count(y)
```

Criterion

the pair that most improves corpus likelihood

Direction

bottom-up: grow from characters

Marking

continuation pieces prefixed with ##

Used by

BERT, DistilBERT, ELECTRA

Favours pairs whose parts are rare alone but nearly always occur together.

Unigram LM

Prune by loss

```
drop pieces with smallest
loss in corpus likelihood
```

Criterion

remove the pieces the corpus needs least

Direction

top-down: shrink a large candidate set

Marking

keeps a probability per piece

Used by

T5, mT5, ALBERT, XLNet

Pieces carry probabilities, so one word has several scored segmentations.

Same output shape, different objective: count pieces, score pieces, or prune what you can afford to lose.

Who decides the tokens? Rules vs corpus statistics

Tokenization can encode human-made boundary rules or learn reusable units from a corpus.

RULE-BASED TOKENIZATION

We specify where boundaries and exceptions should be.

```
"There's"  →  There | 's
```

Decision source	hand-written rules / linguistic conventions
Typical tools	str.split · regex · NLTK Treebank · spaCy rules
Strength	transparent, controllable, often fast
Trade-off	edge cases require more rules and testing

DATA-DRIVEN SUBWORD TOKENIZATION

The tokenizer is fitted to a corpus and learns reusable pieces.

```
lower  →  low | er
```

Decision source	frequency / statistics of character or byte sequences
Typical methods	BPE · WordPiece · SentencePiece-style tokenization
Strength	reuses pieces for rare or unseen words
Trade-off	must be trained / fitted; exact pieces depend on the corpus

Core difference: rules encode our assumptions; data-driven tokenizers learn a vocabulary of reusable pieces from corpus statistics.

Exercise: preprocess a messy policy message

```
URGENT: The Ministry’s 2nd briefing—re-scheduled for 17/09/2026 at 18:00!!!  Email
press.office@gov.example  #ClimatePolicy
```

EXTRACT Which structured fields can regex identify?

TOKENIZE What should stay together: date, time, email, hashtag, re-scheduled?

NORMALIZE What would you lowercase, standardize, or leave untouched?

TEST Invent one variant that would break your rules.

4 min in pairs → 3 min debrief: compare pipelines, not just regex syntax.

Keep this mental model

Hertie School

RAW TEXT
what arrived

PATTERNS
what we can detect

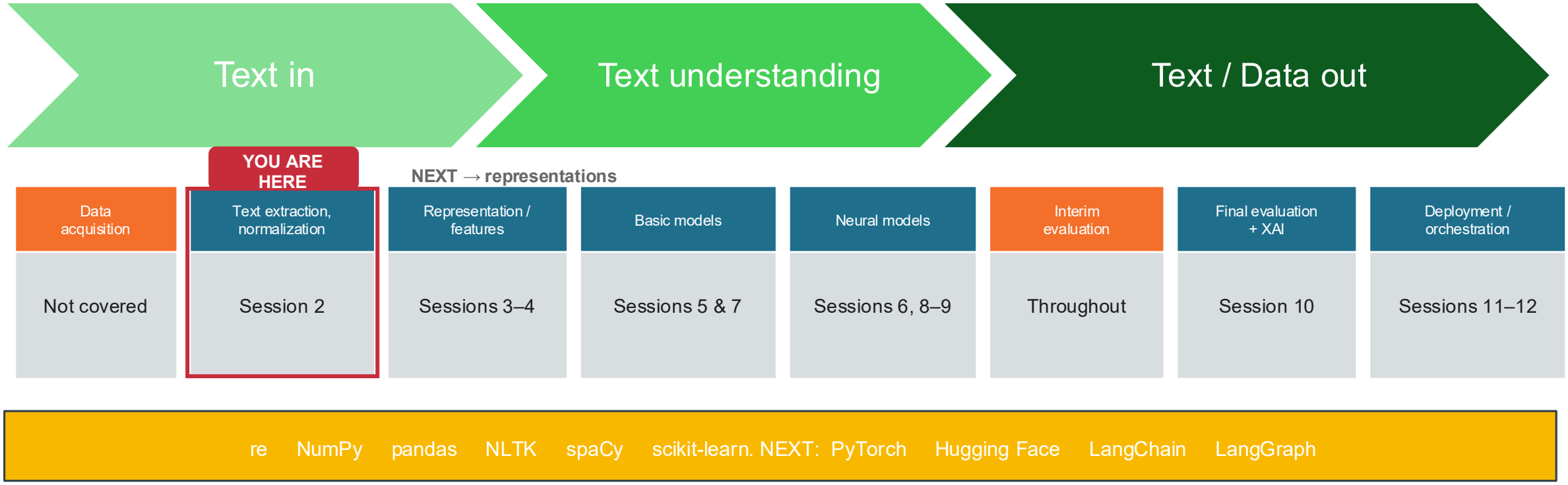
TOKENS
what counts as a unit

NORMALIZED TOKENS
what distinctions we keep

1. Regex is a compact language for matching and transforming string patterns.
2. Tokenization and normalization are task-dependent choices, not neutral cleanup.
3. Every preprocessing rule should be tested against both what it catches and what it misses.

Next session → turning tokens into vectors: one-hot · bag-of-words · TF–IDF

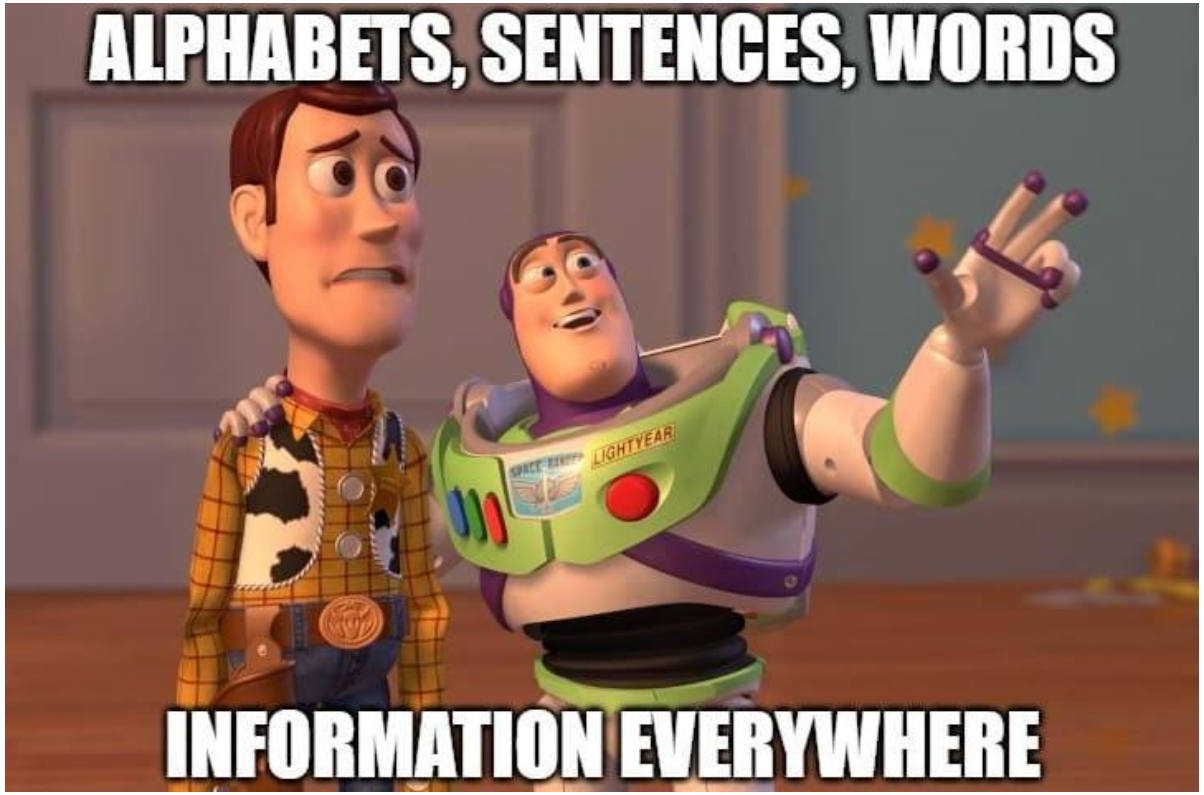
Where we are in the NLP pipeline



Today: strings → regex → tokenization → normalization. Next: turn those units into numerical representations.

Quiz reminder: Next session and on paper.

THANK YOU!


ALPHABETS, SENTENCES, WORDS
LIGHTYEAR
INFORMATION EVERYWHERE